Daniel Gomez Solís C23310
María Fernanda Herrera Gutiérrez C13747

31 de octubre, 2024
Tarea Programada II: descripción del problema

# Predicción del mercado de valores

**Objetivo:** Predecir el precio futuro de una acción o índice del mercado (bursátil) en función de datos históricos del mercado y factores adicionales que influyen en el comportamiento de los precios. La precisión de las predicciones son importantes para lograr una buena toma de decisiones a la hora de invertir y gestionar operaciones financieras. Se desea diseñar un modelo predictivo basado en redes neuronales que sea capaz de anticipar el precio futuro de una acción minimizando el margen de error frente a datos reales.

**Naturaleza del Problema:** La predicción del mercado de valores es un problema de optimización NP-duro, debido a la cantidad de factores complejos y no lineales que afectan los precios de las acciones. Estos incluyen:

- ➔ Datos históricos de precios y volumen de transacciones
- ➔ Indicadores técnicos: herramientas de análisis cuántico como medias móviles, índice de fuerza relativa que indican tendencias de mercado
- ➔ Factores externos: noticias, eventos económicos y políticas internacionales

**Formulación del Problema**

- ➔ Entrada:
  - ◆ Datos históricos del mercado: Precios de apertura, cierre, máximo y mínimo de las acciones, así como el volumen de transacciones.
  - ◆ Indicadores técnicos: Variables calculadas a partir de los datos históricos, como medias móviles, bandas de Bollinger, etc.
  - ◆ Factores externos: Información de noticias, anuncios económicos y otros eventos relevantes.
- ➔ Salida:
  - ◆ Un modelo de red neuronal que prediga el precio futuro de la acción en un tiempo determinado minimizando el error en comparación con los datos reales.

**Definición de red neuronal**

Una red neuronal es un modelo computacional inspirado en la estructura y función del cerebro humano, diseñado para reconocer patrones en datos. Se compone de capas de neuronas interconectadas, donde cada conexión tiene un peso que se ajusta durante el entrenamiento. Conceptos claves para su entendimiento:

- ➔ **Hiperparámetros**: Son configuraciones del modelo que se establecen antes del entrenamiento y no se aprenden a partir de los datos. Se ajustan para optimizar el rendimiento del modelo, incluyendo la tasa de aprendizaje y el número de capas/neuronas.
- ➔ **Funciones de Activación**: Definen cómo se procesan las señales dentro de las neuronas.
- ➔ **Función de Pérdida**: Evalúa el rendimiento del modelo durante el entrenamiento, guiando el ajuste de hiperparámetros.

Ejemplo de utilización de red neuronal:

Se quiere predecir el precio de una acción. Se inicia alimentando la red con datos históricos de precios como entrada, y sus precios futuros como salida. La red ajusta los pesos de las conexiones entre neuronas a través de la retropropagación lo que minimiza el error en las predicciones mediante la función de pérdida. Con cada iteración la red mejora la precisión ya que aprende patrones y relaciones con los datos históricos.

Para abordar esta complejidad, se usarán tres enfoques:

- **Fuerza bruta:** Consiste en evaluar todas las posibles combinaciones de parámetros para un modelo de red neuronal, con el fin de encontrar la mejor configuración en términos de precisión. Se usará para establecer una referencia ya que no es viable para configuraciones grandes. La fuerza bruta sirve como punto de comparación para las soluciones heurística y metaheurística.

  Pasos de fuerza bruta:

  - ➔ Definición del Espacio de Búsqueda: Se identifican todos los hiperparámetros que se desean optimizar y sus posibles valores.
  - ➔ Generación de Combinaciones: Se crean todas las combinaciones posibles de estos hiperparámetros.
  - ➔ Entrenamiento: Para cada combinación se entrena el modelo de red neuronal utilizando datos históricos.
  - ➔ Evaluación: Se mide el rendimiento del modelo (precisión de las predicciones) para cada combinación.
  - ➔ Comparación: Se realizan comparaciones de los resultados para identificar la combinación de parámetros que produce el mejor rendimiento.
  - ➔ Referencia: La combinación final se utiliza de referencia para evaluar los enfoques heurísticos y metaheurísticos.

- **Heurística:** Se utiliza la optimización bayesiana aplicando un modelo probabilístico, como un proceso gaussiano, para ajustar los hiperparámetros de una red neuronal. Este enfoque evalúa el rendimiento de diferentes configuraciones permitiendo predecir el impacto de nuevos parámetros en la función objetivo. A través de una función de adquisición se seleccionan iterativamente los mejores hiperparámetros, minimizando la cantidad de evaluaciones necesarias y optimizando la precisión del modelo en la predicción de precios en el mercado de valores.

  Pasos de la optimización bayesiana:

  - ➔ Inicialización: Se inicia con un conjunto inicial de configuraciones de hiperparámetros y evalúa su rendimiento.
  - ➔ Modelo Probabilístico: Se ajusta un modelo (como un proceso gaussiano) a los datos de rendimiento.
  - ➔ Función de Adquisición: Se define una función que evalúe la calidad de nuevas configuraciones y sugiere nuevos puntos a explorar.
  - ➔ Iteración: Se seleccionan nuevos hiperparámetros basados en la función de adquisición, se evalúa su rendimiento y se actualiza el modelo.
  - ➔ Convergencia: Se repite hasta que se alcance una precisión aceptable o se cumpla un límite de iteraciones.

- **Metaheurística:** El algoritmo genético permite una búsqueda eficiente de soluciones óptimas utilizando procesos de selección y recombinación. Los algoritmos genéticos combinan principios de búsqueda aleatoria y sistemática para explorar soluciones óptimas en espacios de búsqueda amplios y complejos. Estos imitan la selección natural, utilizando una "población" inicial de configuraciones que se cruza y muta a lo largo de varias generaciones. Este método permite que el modelo escape de óptimos locales y refine los hiperparámetros de manera más eficiente y efectiva que la búsqueda exhaustiva o la heurística.
  Pasos del algoritmo genético:

- ➔ Inicialización: Se genera una población inicial de configuraciones de hiperparámetros de la red neuronal, como la tasa de aprendizaje, número de capas ocultas, y neuronas por capa.
- ➔ Evaluación: Se entrena y evalúa cada configuración en la predicción de precios, usando un conjunto de datos históricos.
- ➔ Selección: Se seleccionan las mejores configuraciones según la precisión de las predicciones.
- ➔ Cruce: Se combinan configuraciones "padre" (tasa de aprendizaje de uno y capas de otro) para crear "hijos" con características combinadas.
- ➔ Mutación: Se introducen cambios menores a algunos hijos para explorar variaciones adicionales.
- ➔ Repetición: Se sustituye la población inicial con las nuevas configuraciones y repite el proceso de evaluación, selección, cruce y mutación hasta que se alcance la precisión deseada o un número límite de iteraciones.

**Ejemplo de código base (pseudocódigos)**

1. Fuerza bruta

```
Definir hiperparámetros: tasas_de_aprendizaje, neuronas
mejor_modelo = None
mejor_puntuación = infinito

Para cada combinación en Producto(hiperparámetros):
      modelo = CrearModelo(combinación)
      EntrenarModelo(modelo, datos)
      puntuación = EvaluarModelo(modelo, datos)

Si puntuación < mejor_puntuación:
      mejor_puntuación = puntuación
      mejor_modelo = modelo
Retornar mejor_modelo
```

2. Optimización bayesiana (Heurística)

```
Definir espacio_de_búsqueda: neuronas, tasa_de_aprendizaje
Inicializar configuraciones
Modelo probabilístico =
      AjustarModeloProbabilístico(configuraciones)

Mientras no converger:
    nueva_configuración = SeleccionarConfiguración(Modelo
            probabilístico)
    rendimiento = EvaluarModelo(nueva_configuración, datos)
               ActualizarModeloProbabilístico(nueva_configuración,
rendimiento)
Retornar mejor_configuración
```

3. Algoritmo genético (Metaheurística)

```
Inicializar población de configuraciones

Mientras no alcanzar criterio de parada:
    Evaluar cada individuo en la población
    Seleccionar los mejores individuos
    Para cada pareja de padres:
        hijo = Cruzar(padre1, padre2)
        Si aleatorio < probabilidad_mutación:
            Mutar(hijo)
    Reemplazar población con hijos

Retornar mejor_individuo
```